

Open Tasks — Require Your Action

Project: AI Platform for Managing & Maximizing Job Search

All the code for the 5 build phases has been written and tested (automated tests + simulations), but the items below require access to your own accounts (Supabase, Google Cloud, Railway, etc.), so I couldn't complete them myself. The list is grouped by topic, not a strict chronological order — but starting with Supabase is recommended, since everything else depends on it.

1. Supabase

The foundation for the whole system — needed first.

- [] Create a new Supabase project in the cloud (supabase.com).
- [] Retrieve SUPABASE_URL, the anon key, the service_role key, and the JWT secret (Settings → API).
- [] Run the 4 migration files in supabase/migrations (via **supabase db push**, or paste them manually into the SQL Editor).
- [] Review supabase/config.toml and apply the Authentication settings in the dashboard (password policy, JWT length) if not managing via the CLI.
- [] Confirm a Storage bucket named **resumes** was created (should happen automatically from the migration).
- [] Enable the MFA (TOTP) provider under Authentication → Providers — required before any Super Admin can log in.

2. Google Cloud — Google Sign-In & Gmail

The same Google Cloud project serves both login and the Gmail connection.

- [] Create a project in Google Cloud Console.
- [] Configure the OAuth consent screen in Testing mode, and add your test user emails (up to 100).
- [] Create an OAuth Client ID (Web application) with Supabase's redirect URI.
- [] Enter the Client ID/Secret into Supabase Auth settings (Authentication → Providers → Google).
- [] Enable the Gmail API and add the **gmail.readonly** scope to the consent screen.
- [] Add **{AI_SERVICE_URL}/gmail/connect/callback** as an additional redirect URI on the same client.
- [] Set GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET as environment variables on the ai-service.

3. OpenAI

- [] Provide an OPENAI_API_KEY for the AI agents (ai-service/.env).

4. Railway — hosting the AI service

- [] Create a Railway project for ai-service (a Dockerfile is already included in the code).
- [] Set all environment variables from ai-service/.env.example, including COOKIE_SECURE=true and the real URLs.
- [] Create a second service (Cron Job) from the same repo to run **python -m app.jobs.sync_gmail** on schedule **0 6,18 * * *** (twice daily).

5. Frontend hosting — not addressed at all yet

A step that hasn't been discussed or done explicitly so far.

- [] Choose a static hosting provider (e.g. Vercel, Netlify, Cloudflare Pages).
- [] Connect the web/ folder for deployment, and set the required build environment variables (VITE_SUPABASE_URL, etc.).
- [] Update FRONTEND_URL on the ai-service (and its CORS setting) to the site's final URL.

6. User management & security

- [] Manually promote the first user(s) to the **super_admin** role (and **support** if needed) via the Supabase SQL Editor.
- [] Complete the 2FA enrollment flow (scan the QR code) for each Super Admin, on their first login.

7. Testing against real infrastructure

So far, all testing has been automated (mocked) or local simulation only.

- [] Run through every manual checklist in the README (Phases 1–5) against the real environment once it's configured.
- [] Fully verify the OAuth flows (Google login, Gmail connect, 2FA) against real accounts.

8. Git

A local repository is already connected to GitHub with one commit ("first commit") that wasn't made by me — none of Phases 1–5's work has been committed yet.

- [] Decide whether and how to commit and push the existing work (not done on my own initiative, per policy).

For full details on each phase, see the README and the project's plan file.